Minería de Datos

Clasificación

Algoritmos (**SVM, bagging, boosting**)

Valentin Barriere / Mauricio Cerda / Cinthia Sánchez

CC5205, Otoño 2026

Basado en las slides de Bárbara Poblete

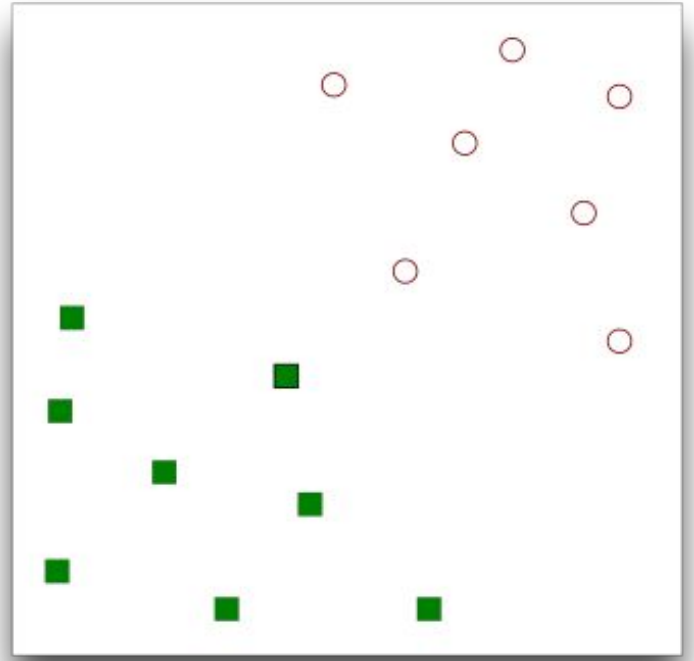
Support Vector Machines

Se formula el problema de clasificación como un problema de optimización.

Encontrar un hiperplano lineal (decision boundary).

- Representa una frontera de decisión usando un subconjunto de ejemplos de entrenamiento, conocidos como support vectors.
- Trabaja bien con datos de alta dimensionalidad y evita la "curse of dimensionality"

Support Vector Machines

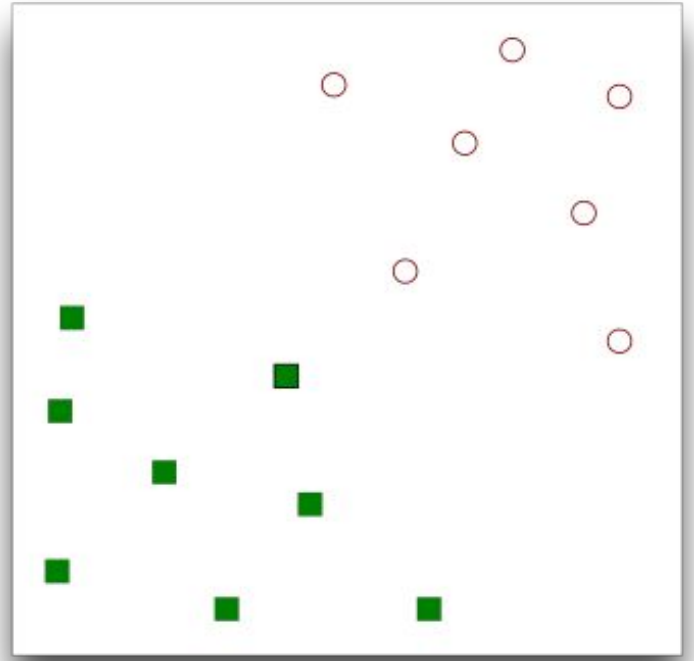


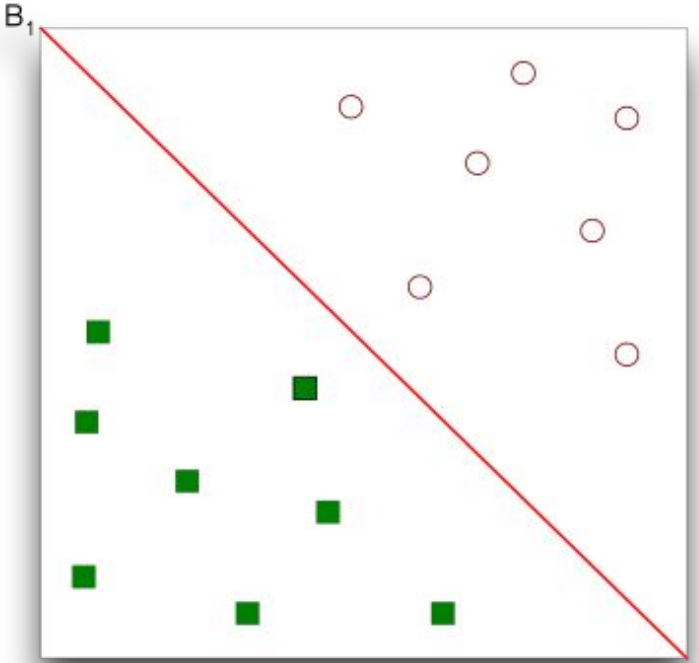
Ejemplo: Tenemos un dataset de n-dimensiones (proyectado a 2).

Podemos decir que estos datos son linealmente separables (separables por un hiperplano)

- Encontrar un hiperplano lineal (*decision boundary*) que separe los ejemplos positivos de los negativos.

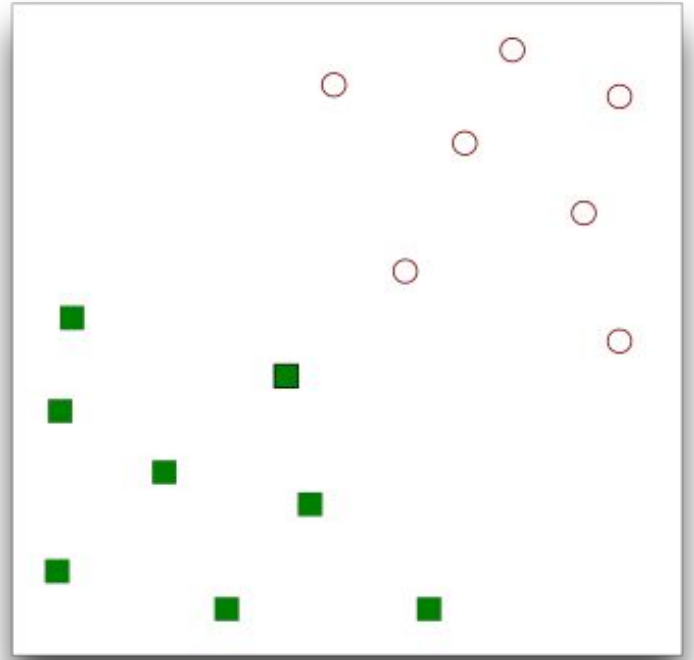
Support Vector Machines

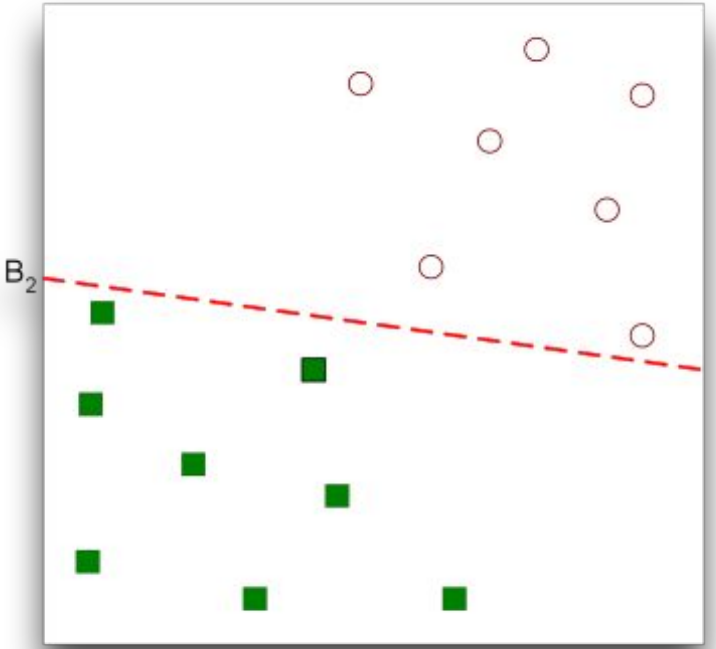




Una posible solución

Support Vector Machines

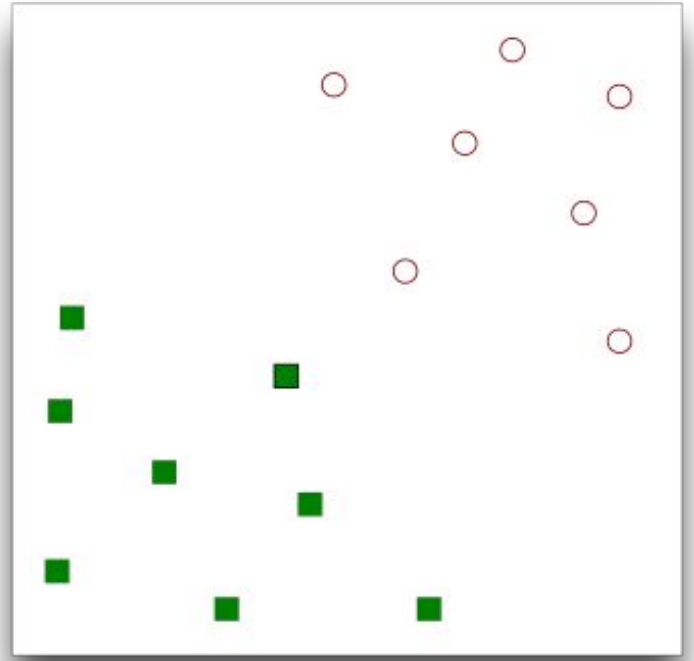


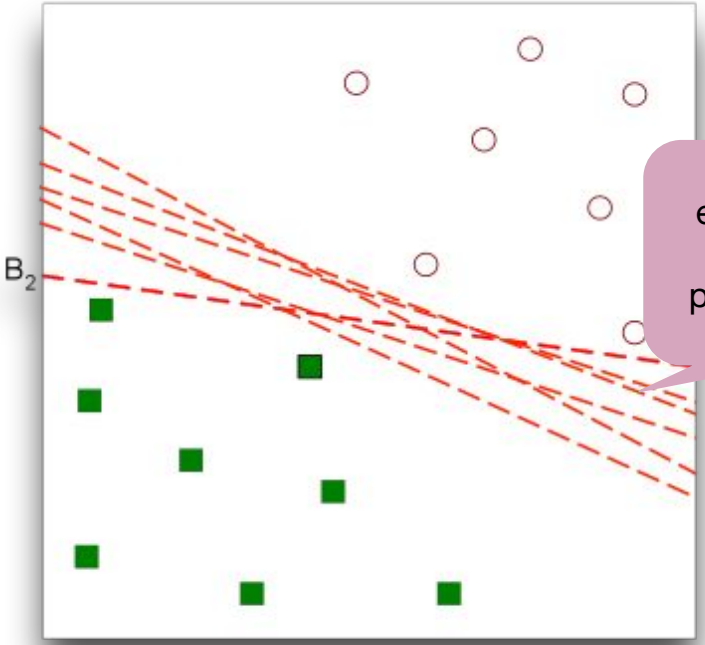


Otra posible solución

Support Vector Machines

Este modelo puede extenderse a datos que no son linealmente separables.

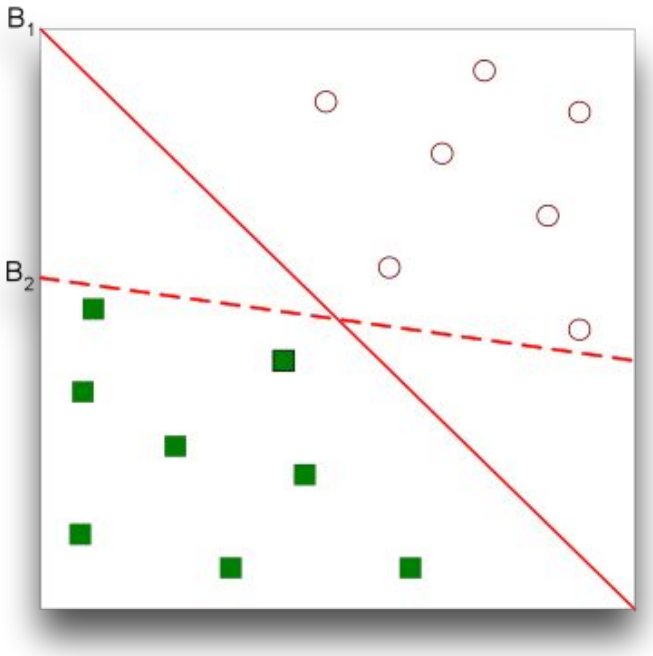




SVM se basa en aprender a encontrar el plano marginal maximal

¡Infinitas soluciones!

Support Vector Machines



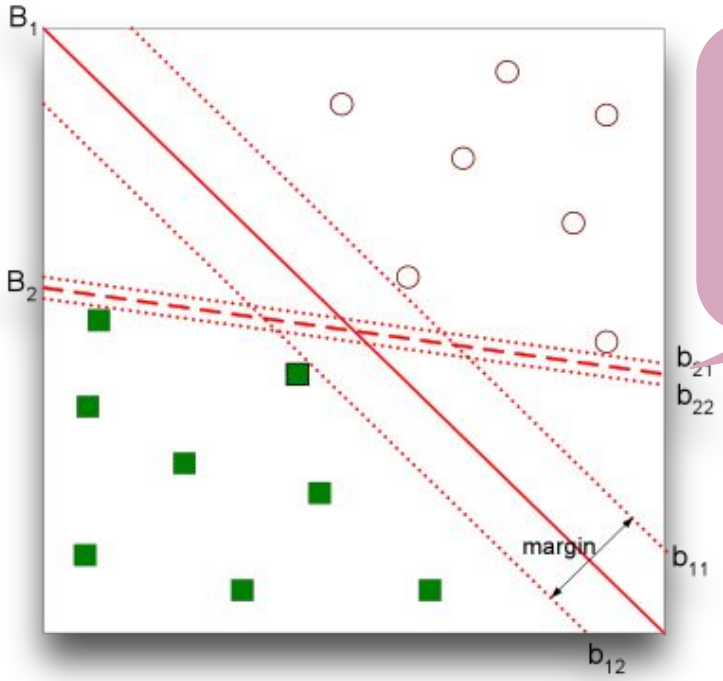
¿Cuál es mejor? ¿B1 o B2?
¿Cómo definimos qué es mejor?

Support Vector Machines

Encontrar un hiperplano que **maximice** el margen de entrenamiento (menores errores de generalización)

- El margen del hiperplano es la distancia que hay de los puntos positivos y negativos más cercanos al hiperplano.

Entre más ancho el margen, mayor el poder de generalización.
=> B1 es mejor que B2



Menor margen hace que sea más susceptible a ruido y perturbaciones, lo que puede llevar a overfitting y mala generalización.

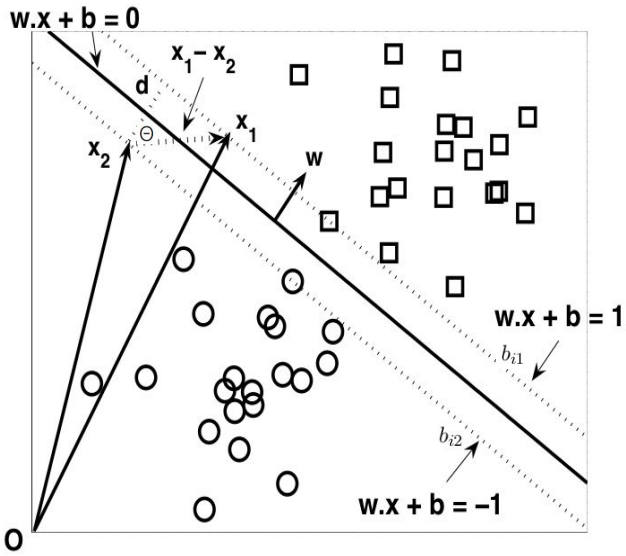
Support Vector Machines

Un SVM lineal es un clasificador que busca un hiperplano con el máximo margen.

- Sea un problema de clasificación binaria con N ejemplos, cada ejemplo se representa como la tupa x_i, y_i (i=1,2,...,N), donde x_i es un vector de d dimensiones $(x_{i1}, x_{i2}, \ldots, x_{id})^T$ e $y_i \in \{-1, 1\}$ (ejemplos negativos y positivos).
- El límite de decisión de un clasificador lineal se escribe de la siguiente manera:

$$\mathbf{w} \cdot \mathbf{x} + b = 0,$$

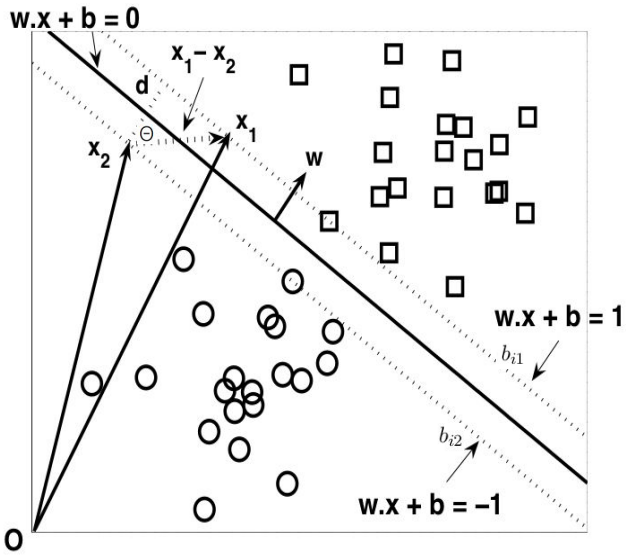
- Donde **w** y b son parámetros del modelo.



Support Vector Machines

Consiste en:

1. Formular el margen en función de los parámetros.
2. Formular un problema de optimización que permita encontrar el hiperplano de máximo margen.



Support Vector Machines

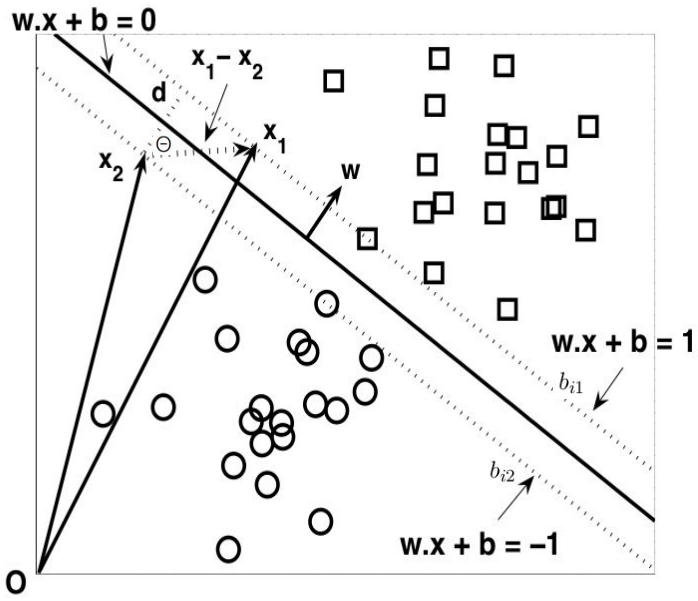
- El **hiperplano w·x+b=0** separa los ejemplos positivos (cuadrados) de los negativos (círculos)
- Cualquier ejemplo que se encuentre en el límite de decisión debe satisfacer la ecuación del hiperplano.
- Por ejemplo, sean x_a y x_b dos puntos localizados en el límite de decisión:

$$\mathbf{w} \cdot \mathbf{x}_a + b = 0,$$
$$\mathbf{w} \cdot \mathbf{x}_b + b = 0.$$

- Si restamos las dos ecuaciones obtenemos:

$$\mathbf{w} \cdot (\mathbf{x}_b - \mathbf{x}_a) = 0,$$

- Cómo x_b-x_a es paralelo al límite de decisión, entonces w es perpendicular al hiperplano (el producto punto de dos vectores ortogonales es cero).



Support Vector Machines

- Para cualquier cuadrado x_s localizado **sobre el límite de decisión**, se cumple que:

$$\mathbf{w} \cdot \mathbf{x}_s + b = k,$$

donde k > 0.

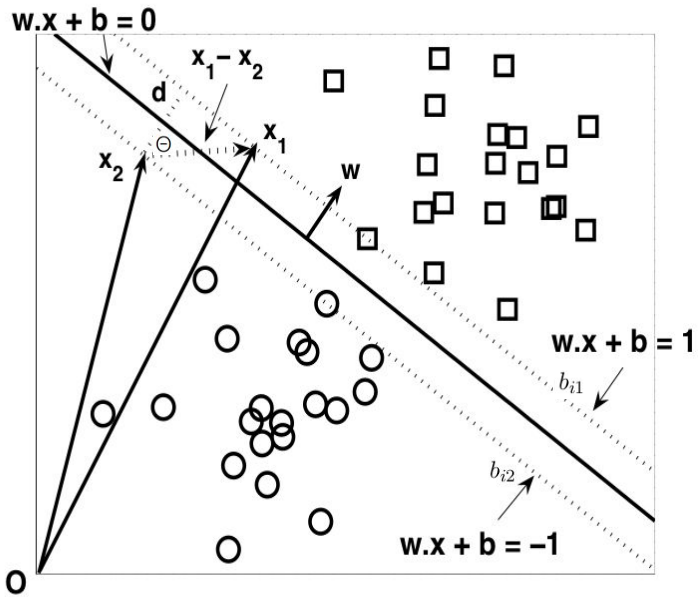
- De manera análoga, para cualquier círculo x_c localizado **bajo el límite de decisión**, se cumple que:

$$\mathbf{w} \cdot \mathbf{x}_c + b = k',$$

done k'<0.

- Si etiquetamos todos los cuadrados como la clase positiva +1 y todos los círculos como la clase negativa -1, podemos predecir la etiqueta **y** para cualquier ejemplo de test **z** de la siguiente manera:

$$y = \begin{cases} 1, & \text{if } \mathbf{w} \cdot \mathbf{z} + b > 0; \\ -1, & \text{if } \mathbf{w} \cdot \mathbf{z} + b < 0. \end{cases}$$



El Margen de un Clasificador Lineal

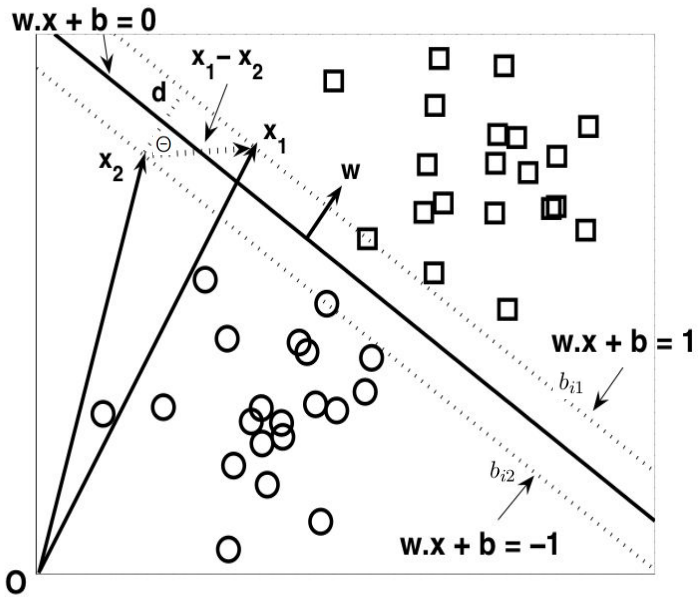
- Podemos re-escalar los parámetros w y b del límite de decisión de tal manera que los hiperplanos paralelos b_{i1} y b_{i2} puedan expresarse de la siguiente manera:

$$b_{i1}: \ \mathbf{w} \cdot \mathbf{x} + b = 1,$$
$$b_{i2}: \ \mathbf{w} \cdot \mathbf{x} + b = -1.$$

- El margen del límite de decisión se calcula como la distancia entre estos dos hiperplanos.
- Calculamos la distancia del cuadrado x_1 y el círculo x_2. Esto se hace sustituyendo x_1 y x_2 en las ecuaciones de b_{i1} y b_{i2} respectivamente. Lo que nos da:

1) $\mathbf{w \cdot x_1 + b = 1}$ y 2) $\mathbf{w \cdot x_2 + b = -1}$.

- Si sustraemos la segunda ecuación de la primera obtenemos $\mathbf{w \cdot (x_1 - x_2) = 2}$



El Margen de un Clasificador Lineal

- El producto punto de dos vectores (a·b) se puede representar como la norma ||a||*||b||*cos(Θ). Es decir, multiplicar sus magnitudes (normas) por el coseno del ángulo que forman entre ellos.

- Tenemos entonces que $||w||*||x_1-x_2||*\cos(\Theta)=2$

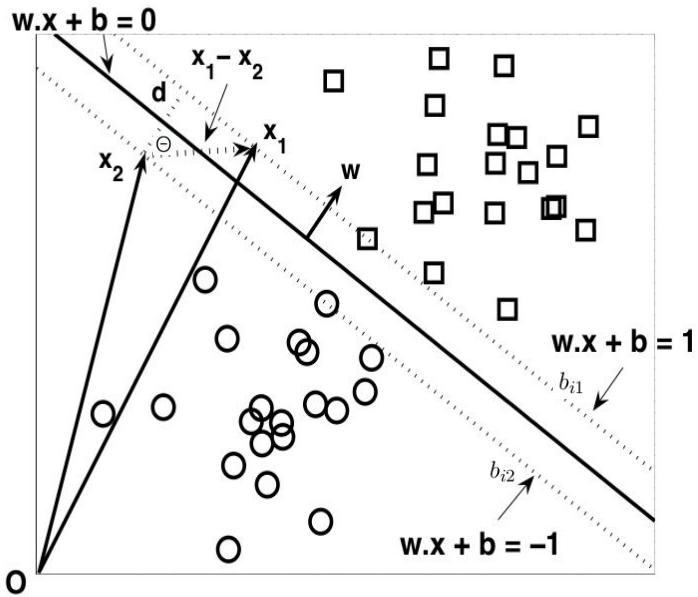
- Si miramos la figura podemos ver que $||x_1-x_2||*\cos(\Theta) = d$.

- Entonces

$$||\mathbf{w}|| \times d = 2$$

$$\therefore d = \frac{2}{||\mathbf{w}||}.$$

Queremos maximizar este margen



¡ Encontramos una expresión del margen que depende de w !

Aprendiendo una SVM Lineal

La fase de **entrenamiento de una SVM lineal** implica la estimación de los parámetros w y b del límite de decisión a partir de los datos de entrenamiento.

Los parámetros deben elegirse de manera que se cumplan las dos siguientes condiciones:

$$\mathbf{w} \cdot \mathbf{x_i} + b \geq \ 1 \text{ if } y_i = 1,$$
$$\mathbf{w} \cdot \mathbf{x_i} + b \leq -1 \text{ if } y_i = -1.$$

- Estas condiciones imponen el requisito de que todas las instancias de entrenamiento de la clase y = 1 (los cuadrados) deben estar situadas en o **sobre el hiperplano w·x+b = 1**.
- Mientras que las instancias de la clase y = -1 (los círculos) deben estar situadas **en o debajo del hiperplano w·x+b = -1**.

Aprendiendo una SVM Lineal

- Maximizar el margen equivale a minimizar la siguiente función objetivo:

$$\boxed{f(\mathbf{w}) = \frac{\|\mathbf{w}\|^2}{2}.}$$

$$\max_{w} \frac{2}{||w||} \Leftrightarrow \min_{w} \frac{||w||^2}{2}$$

- La tarea de aprendizaje en SVM puede formalizarse como el siguiente problema de optimización

$$\min_{\mathbf{w}} \frac{\|\mathbf{w}\|^2}{2}$$

$$\text{subject to} \qquad y_i(\mathbf{w} \cdot \mathbf{x_i} + b) \geq 1, \quad i = 1, 2, \ldots, N.$$

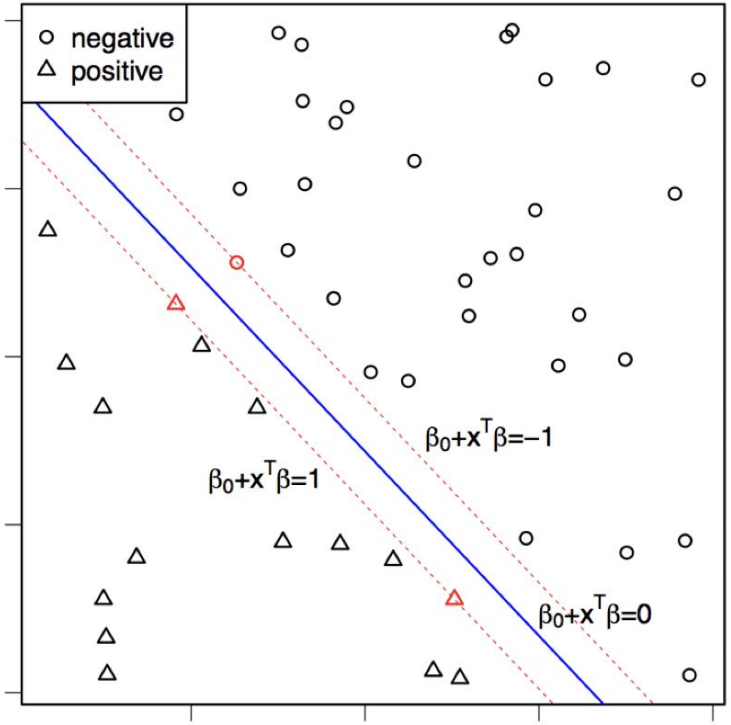
- Dado que la función objetivo es cuadrática y las restricciones son lineales para los parámetros w y b, esto se conoce como un **problema de optimización convexa**.

Hiperplano óptimo: puntos de soporte

- El hiperplano está definido por una combinación lineal de puntos en el borde del margen (puntos de soporte).
- ¡La solución sólo depende de los puntos de soporte y no de todos los datos!
- Por lo que podemos escribir

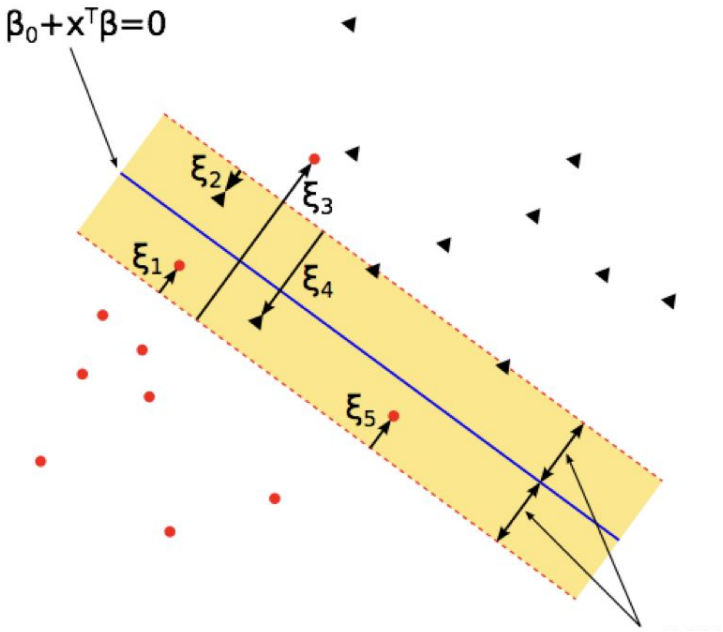
$$\beta = x^T \alpha$$

- donde $\alpha_i = 0$ cuando no es un punto de soporte.



Aprendiendo una SVM Lineal (más realista)

- Para hacerlo "más realista" (Soft Margin) porque los datos casi nunca son perfectamente separables, introducimos lo siguiente.
- Introducir para cada ejemplo una variable auxiliar (de Holgura) $\xi_i \geq 0$
- ξ_i cantidad relativa, respecto al margen, por la cual la predicción está en el lado incorrecto
- Si la clasificación es correcta $\xi_i = 0$
- Si $0 < \xi_i \leq 1$ el punto está entre el margen y el lado correcto
- Clasificaciones incorrectas ocurren si $\xi_i > 1$



Aprendiendo una SVM Lineal (más realista)

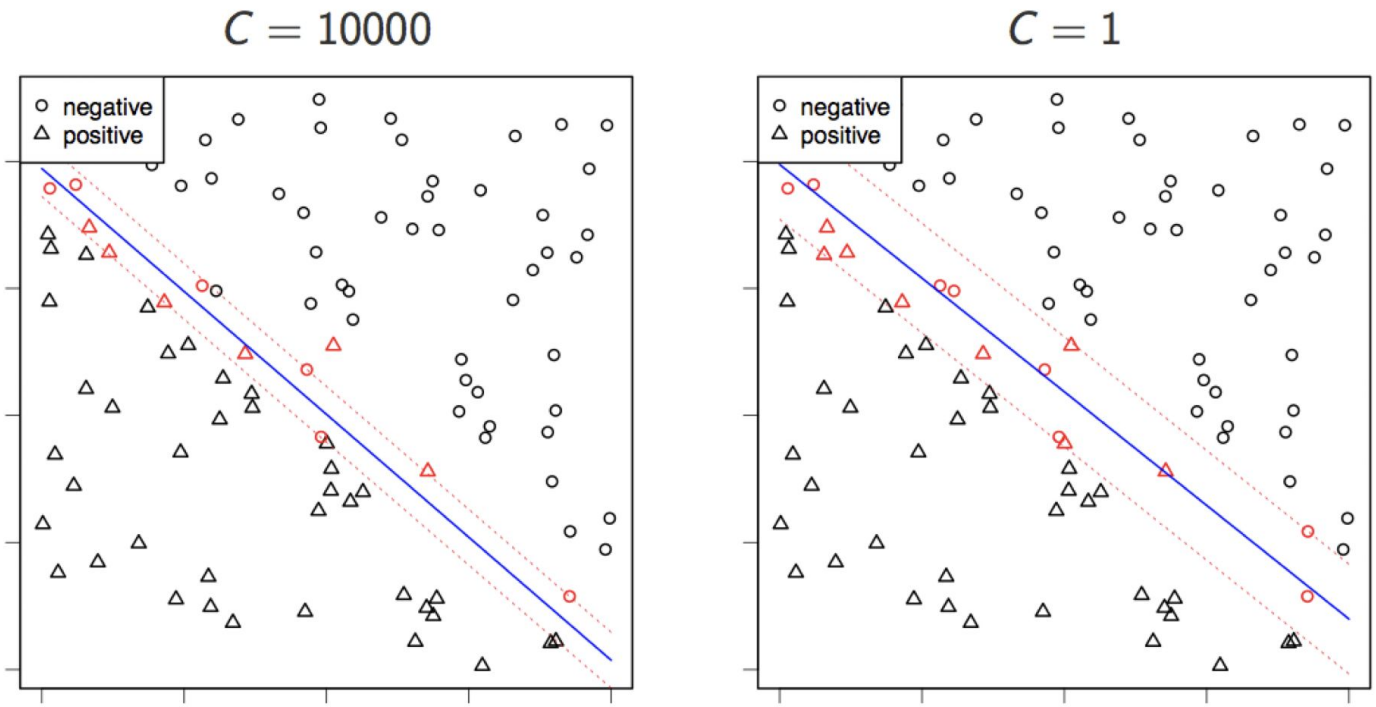
$$\min_{\beta_0,\beta} \frac{1}{2}|\beta|^2 + C\sum_{i=1}^{n} \xi_i$$

$$\text{sujeto a } \xi_i \geq 0, y_i(\beta_0 + x^T\beta) \geq 1 - \xi_i$$

- El parámetro C > 0 representa el trade-off entre la penalización de la variable auxiliar y el margen.

- Si $C = \infty$ el resultado es equivalente al hiperplano óptimo
  - C muy alto: el modelo hará el margen muy estrecho con tal de minimizar errores.
  - C bajo: prefiere una frontera más ancha y suave que generalice mejor.

Aprendiendo una SVM Lineal (más realista)

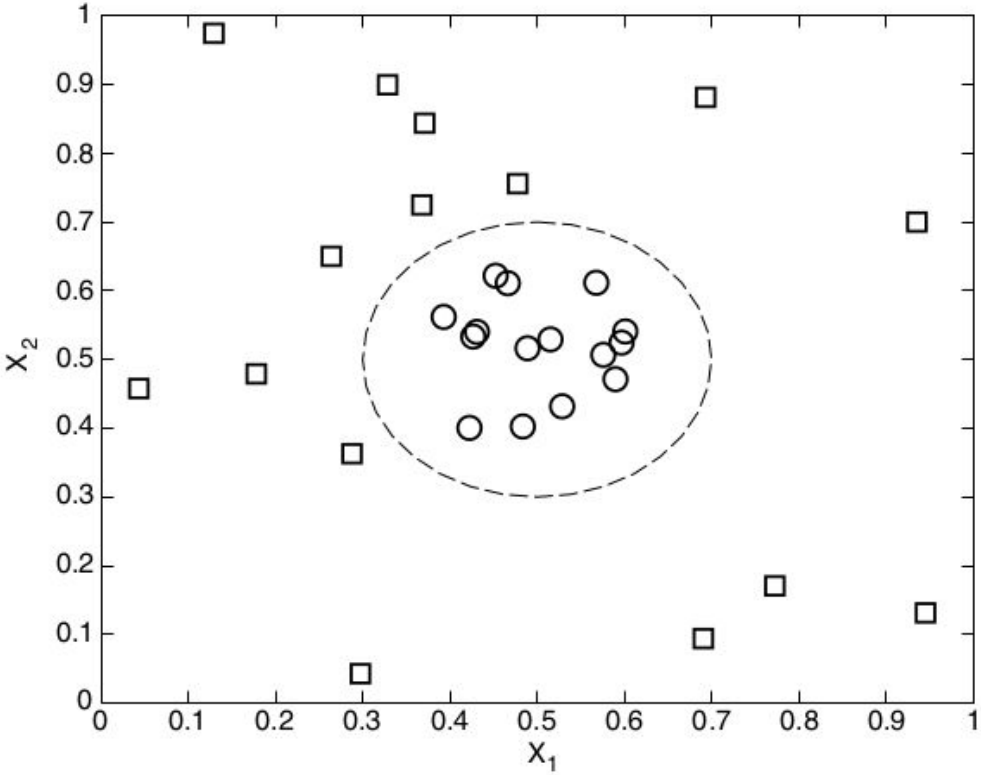


Support Vector Machines No-Lineales

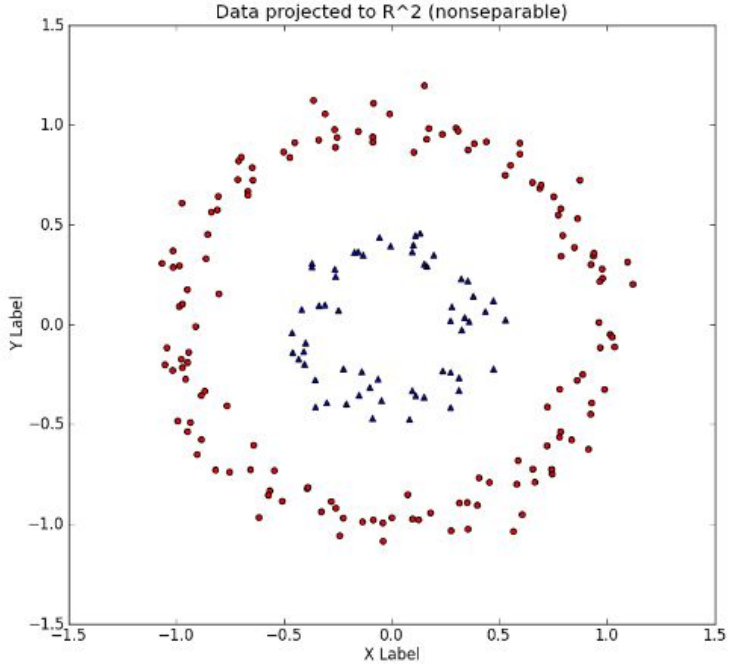
- El diagrama muestra un ejemplo de un dataset bidimensional compuesto por cuadrados (y = 1) y círculos (y = -1).

- Todos los círculos están agrupados cerca del centro del diagrama y todos los cuadrados se distribuyen más lejos del centro.

- Este problema no se puede resolver con una SVM lineal.



Support Vector Machines No-Lineales



¿qué transformación podríamos hacer para que este problema sea linealmente separable?

source: http://www.eric-kim.net/eric-kim-net/posts/1/kernel_trick.html

Support Vector Machines No-Lineales

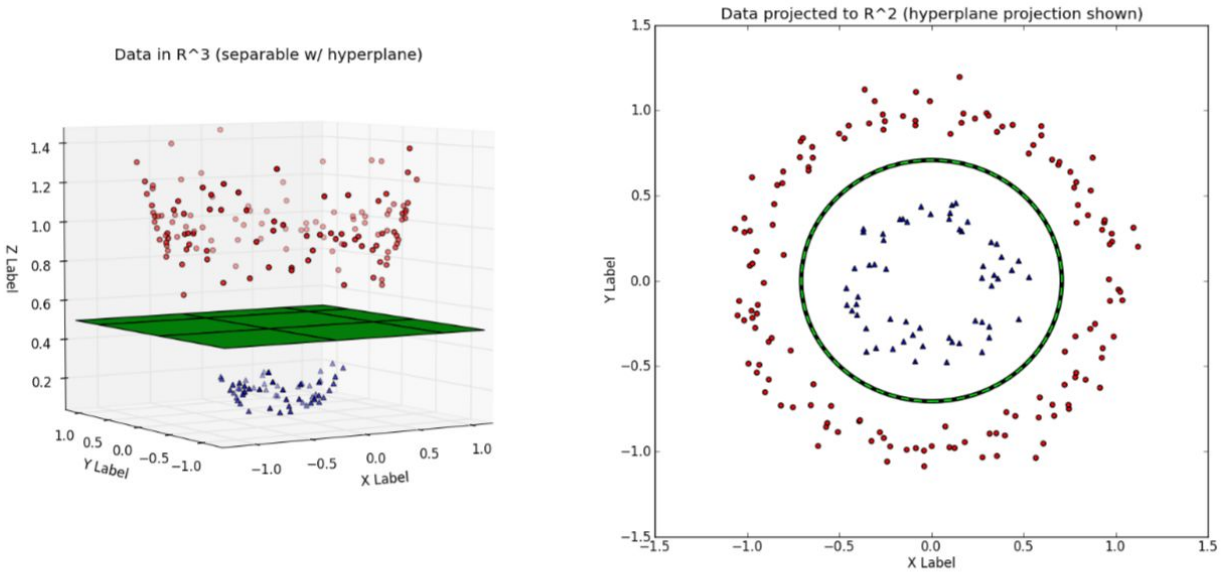


Figure 6: (Left) The decision boundary $\vec{w}$ shown to be linear in $\mathbb{R}^3$. (Right) The decision boundary $\vec{w}$, when transformed back to $\mathbb{R}^2$, is nonlinear.

source: http://www.eric-kim.net/eric-kim-net/posts/1/kernel_trick.html

Support Vector Machines No-Lineales

Podríamos aplicar una **transformación no lineal Φ** para mapear los datos de su espacio de original a un nuevo espacio (de más dimensiones) donde el límite de decisión se vuelva lineal.

Supongamos que elegimos la siguiente transformación que transforma de 2 dimensiones a 5 dimensiones:

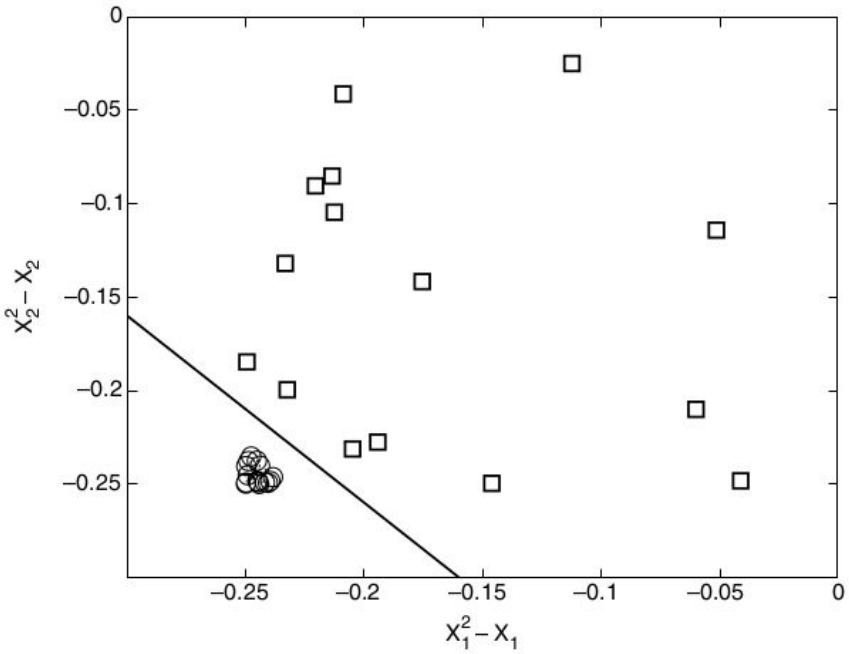
$$\Phi : (x_1, x_2) \longrightarrow (x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, 1).$$

En este espacio transformado si es posible encontrar parámetros $\mathbf{w} = (w_0,\ w_1,\ \ldots,\ w_4)$ que separen linealmente los datos:

$$w_4 x_1^2 + w_3 x_2^2 + w_2\sqrt{2}x_1 + w_1\sqrt{2}x_2 + w_0 = 0.$$

Support Vector Machines No-Lineales

- La figura muestra que en el espacio transformado se puede construir un límite de decisión lineal para separar las clases.
- Un problema potencial de este enfoque es que puede sufrir de la **maldición la dimensionalidad**: para datos de alta dimensión muchas técnicas de data mining no escalan o no funcionan bien.
- Mostraremos cómo una SVM no lineal evita este problema usando un truco llamado **Kernel Trick**.



SVM kernel trick

- Calcular explícitamente la transformación no-lineal puede ser costoso.

- Sin embargo recordar que, β es una combinación lineal de los puntos de soporte, ie $\beta = x^T \alpha$ y $\beta_j = \sum_{i=1}^{n} \alpha_i x_{ij}$ donde $\alpha = 0$ si x_i no es de soporte.
- El hiperplano separatriz se puede formular como:

$$f(\mathbf{x}_0) = \beta_0 + \sum_{j=1}^{m} x_{0j} \beta_j = \beta_0 + \sum_{i=1}^{n} \alpha_i \mathbf{x}_i^T \mathbf{x}_0$$

- Aplicando la transformación

$$f(\mathbf{x}_0) = \beta_0 + \sum_{i=1}^{n} \alpha_i \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_0)$$

SVM kernel trick

- Definición (función kernel)

$$K(x, x') = \phi(x)^T \phi(x')$$

- Definición (kernel SVM)

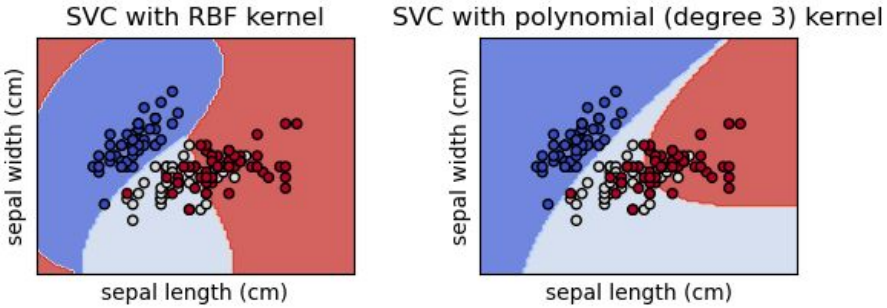
$$f(x_0) = \beta_0 + \sum_{i=1}^{n} \alpha_i K(x_i, x_0)$$

- El “truco del kernel”: no se requiere transformar los datos.

- ¡No se requiere una forma explícita para ϕ !

Support Vector Machines No-Lineales

- Se pueden especificar diferentes funciones de kernel para la función de decisión. Se proporcionan kernels comunes, pero también es posible especificar kernels personalizados.
  - Polinomial
  - Exponencial
  - Radial Basis Function (RBF)
  - etc.



Resumen SVM

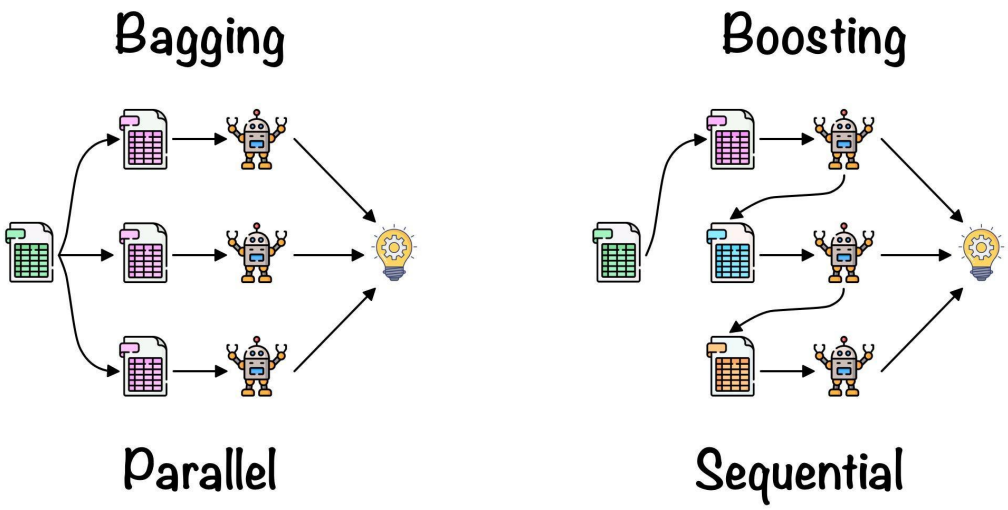
- La formulación de SVM presentada en esta clase se limita a problemas de clasificación binaria. Existen adaptaciones para trabajar con múltiples clases.
- El problema de aprendizaje de una SVM se formula como un problema de optimización convexa en donde hay algoritmos eficientes para encontrar el óptimo global. Otros métodos de clasificación como los árboles de decisión y las redes neuronales tienden a encontrar óptimos locales.
- La SVM optimiza explícitamente la capacidad de generalización al maximizar el margen del límite de decisión.
- En una SVM el usuario debe ajustar hiper-parámetros, como el tipo de función de Kernel y el costo C para las variables de holgura (esto puede ser caro).
- La gran limitación de las SVMs es que no escalan bien para datasets masivos.

Ejemplo SVM

https://colab.research.google.com/drive/1BBeRlTbVzJMdr6xjMGC2aEh2lDWoUfiL

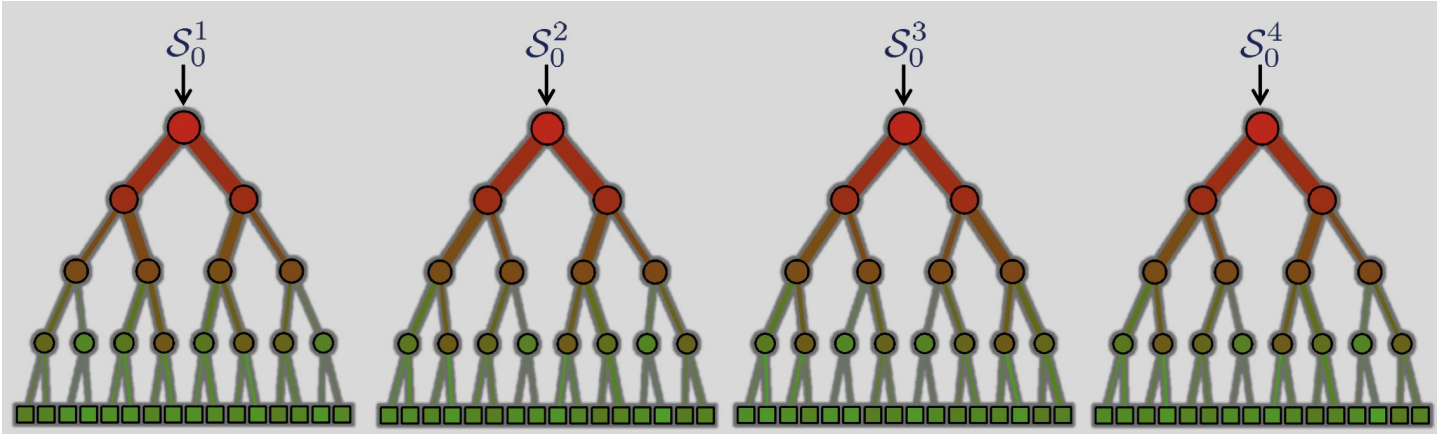
¿Cómo combinar clasificadores?

- Supongamos que tenemos más de un clasificador ¿cómo combinarlos?



Bagging

- Si tuviéramos N clasificadores ligeramente diferentes (no-correlacionados) la clasificación tendría mejor generalización.
- Los árboles son el ejemplo más frecuente.
- En este caso se entrenan N árboles con un subconjunto ligeramente diferente de datos.

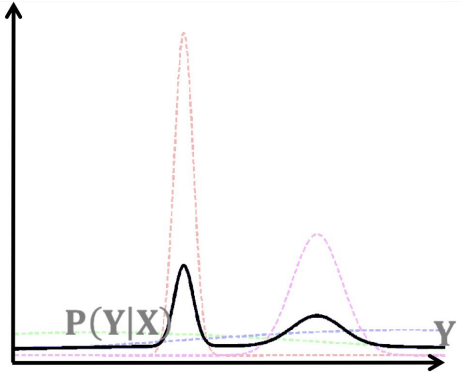


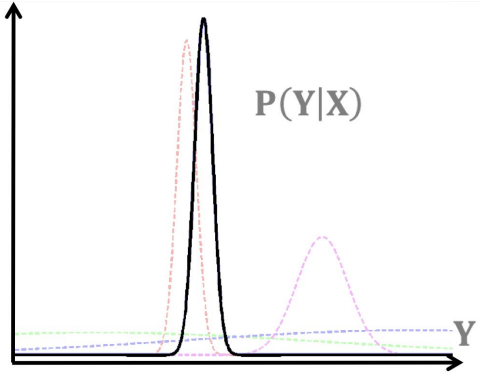
Bagging: votación

- Al terminar el bagging (o boostrap) tendremos múltiples respuestas.

$$\mathbf{P(Y|X)} = \frac{1}{T}\sum_{t=1}^{T} P_t(\mathbf{Y|X})$$

$$\mathbf{P(Y|X)} = \frac{1}{Z}\prod_{t=1}^{T} P_t(\mathbf{Y|X})$$



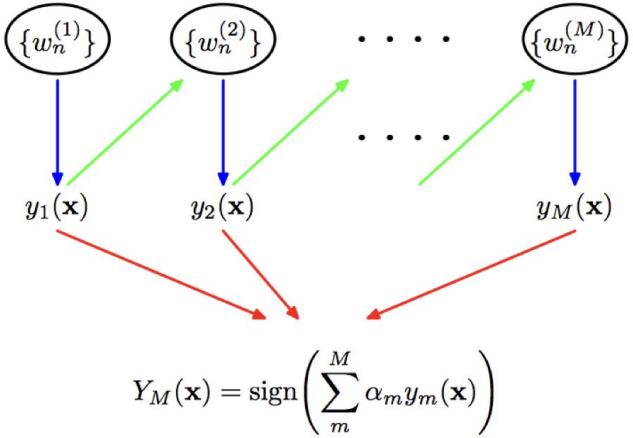


Boosting

- Supongamos que tenemos clasificadores que son sólo ligeramente mejores que una respuesta aleatoria (clasificadores débiles), ¿cómo construir un buen clasificador? (fuerte)

- Idea: entrenar clasificadores en forma secuencial.

Figure 14.1 Schematic illustration of the boosting framework. Each base classifier $y_m(\mathbf{x})$ is trained on a weighted form of the training set (blue arrows) in which the weights $w_n^{(m)}$ depend on the performance of the previous base classifier $y_{m-1}(\mathbf{x})$ (green arrows). Once all base classifiers have been trained, they are combined to give the final classifier $Y_M(\mathbf{x})$ (red arrows).



adaBoosting

- La implementación más conocida de boosting is adaptative boosting (adaBoosting).

- Dado un problema de clasificación binario con vectores de entrada $X_1, ..., X_n$
y su clase asociada $t_1, ..., t_n$

- Al comenzar los pesos a cada dato son iguales, eg w=1/n

adaBoosting

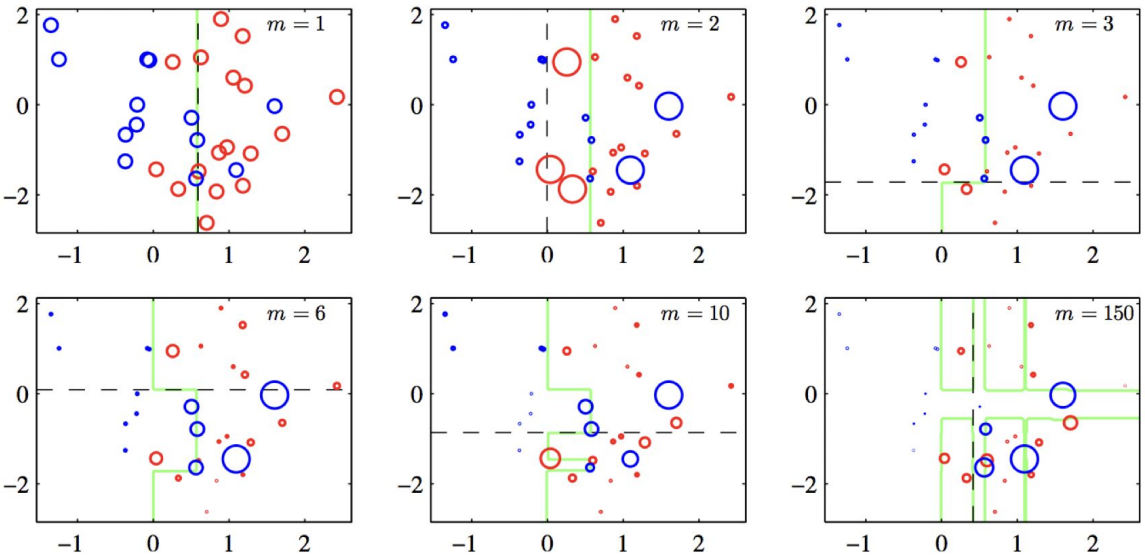


Figure 14.2 Illustration of boosting in which the base learners consist of simple thresholds applied to one or other of the axes. Each figure shows the number m of base learners trained so far, along with the decision boundary of the most recent base learner (dashed black line) and the combined decision boundary of the ensemble (solid green line). Each data point is depicted by a circle whose radius indicates the weight assigned to that data point when training the most recently added base learner. Thus, for instance, we see that points that are misclassified by the $m = 1$ base learner are given greater weight when training the $m = 2$ base learner.

adaBoosting

AdaBoost

1. Initialize the data weighting coefficients $\{w_n\}$ by setting $w_n^{(1)} = 1/N$ for $n = 1, \ldots, N$.
2. For $m = 1, \ldots, M$:
   (a) Fit a classifier $y_m(\mathbf{x})$ to the training data by minimizing the weighted error function

$$J_m = \sum_{n=1}^{N} w_n^{(m)} I(y_m(\mathbf{x}_n) \neq t_n) \tag{14.15}$$

   where $I(y_m(\mathbf{x}_n) \neq t_n)$ is the indicator function and equals 1 when $y_m(\mathbf{x}_n) \neq t_n$ and 0 otherwise.

   (b) Evaluate the quantities

$$\epsilon_m = \frac{\sum_{n=1}^{N} w_n^{(m)} I(y_m(\mathbf{x}_n) \neq t_n)}{\sum_{n=1}^{N} w_n^{(m)}} \tag{14.16}$$

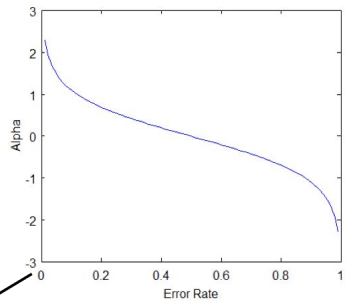
   and then use these to evaluate

$$\alpha_m = \ln\left\{\frac{1-\epsilon_m}{\epsilon_m}\right\}. \tag{14.17}$$

   (c) Update the data weighting coefficients

$$w_n^{(m+1)} = w_n^{(m)} \exp\{\alpha_m I(y_m(\mathbf{x}_n) \neq t_n)\} \tag{14.18}$$

¿Cuándo es mayor α_m?



¿Cómo cambian los pesos?

$w_n^{(m+1)}$

adaBoosting: ejemplo

- Primer algoritmo para detección de caras que alcanza “tiempo real” (1 fps)

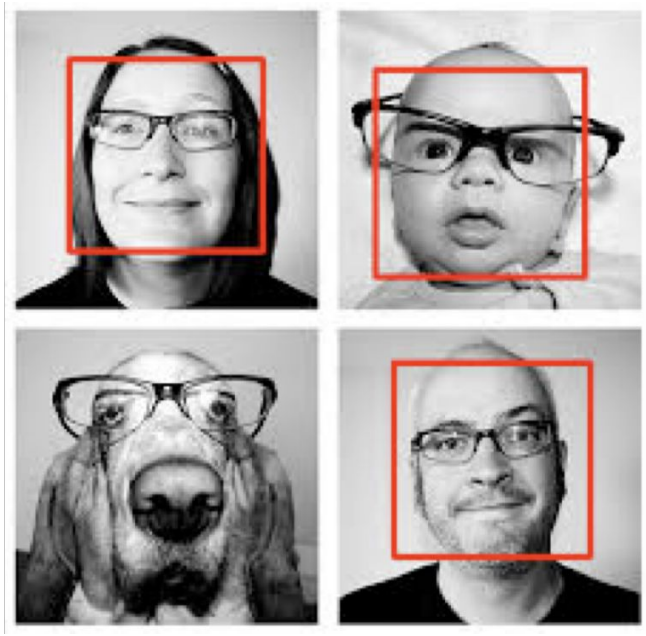
International Journal of Computer Vision 57(2), 137–154, 2004
© 2004 Kluwer Academic Publishers. Manufactured in The Netherlands.

Robust Real-Time Face Detection

PAUL VIOLA
Microsoft Research, One Microsoft Way, Redmond, WA 98052, USA
viola@microsoft.com

MICHAEL J. JONES
Mitsubishi Electric Research Laboratory, 201 Broadway, Cambridge, MA 02139, USA
mjones@merl.com

Received September 10, 2001; Revised July 10, 2003; Accepted July 11, 2003



dcc

CIENCIAS DE LA COMPUTACIÓN
UNIVERSIDAD DE CHILE

www.dcc.uchile.cl